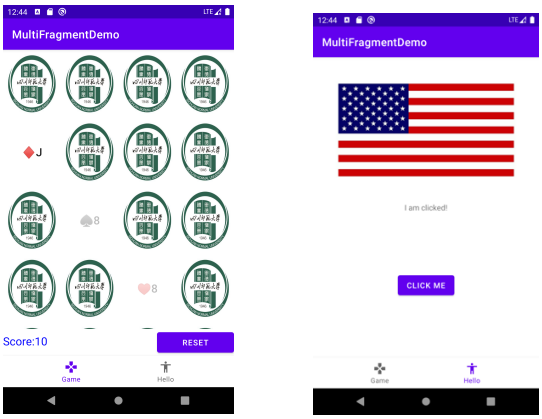
多 Fragment 应用



1. 主 MainActivity

```
class MainActivity : AppCompatActivity() {
    val gameFragment = GameFragment.newInstance()
    val helloFragment = HelloFragment.newInstance()
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        if(savedInstanceState==null) {
            supportFragmentManager.beginTransaction()
                    .replace(R.id.frameLayout, gameFragment)
                    .commit()
        }
        bottomNV.setOnNavigationItemSelectedListener {
            when(it.itemId){
                R.id.game ->
                    supportFragmentManager.beginTransaction()
                        .replace(R.id.frameLayout,gameFragment)
                        .commit()
                R.id.hello ->
                    supportFragmentManager.beginTransaction()
                    .replace(R.id.frameLayout,helloFragment)
                    .commit()
            }
            true
        }
    }
}
```

2. HelloFragment

```
class HelloFragment : Fragment() {

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        // Inflate the layout for this fragment
        return inflater.inflate(R.layout.fragment_hello, container, attachToRoot: false)
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        val button : Button! = view.findViewById<Button>(R.id.button)
        val textView : TextView! = view.findViewById<TextView>(R.id.textView)
        button.setOnClickListener { it: View!
            textView.text = "I am clicked!"
        }
    }

    companion object {
        @JvmStatic
        fun newInstance() : HelloFragment = HelloFragment()
    }
}
```

3. GameFragment

```
class GameFragment : Fragment() {
    val cardButtons : MutableList<Button> = mutableListOf<Button>()
    lateinit var adapter: CardRecyclerViewAdapter
    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        // Inflate the layout for this fragment
        return inflater.inflate(R.layout.fragment_game, container, attachToRoot: false)
    }
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        val rgame : CardMatchingGame? = loadData()
        if (rgame != null) {
            game = rgame
        }else{
            game = CardMatchingGame( count: 24)
        }
        adapter = CardRecyclerViewAdapter(game)
        recylerView.adapter = adapter
        val configuration : Configuration! = resources.configuration
        if (configuration.orientation == Configuration.ORIENTATION_LANDSCAPE) {
            recylerView.layoutManager = GridLayoutManager(activity, spanCount: 6)
        }else{
            val gridLayoutManager = GridLayoutManager(activity, spanCount: 4)
            recylerView.layoutManager = gridLayoutManager
        }
        updateUI()
        adapter.setOnCardClickListener { it: Int
            game.chooseCardAtIndex(it)
            updateUI()
        }
        reset.setOnClickListener { it: View!
            game.reset()
            updateUI()
        }
    }

    fun updateUI() {
        adapter.notifyDataSetChanged()
        score.text = String.format("%s%d","Score:", game.score)
    }
    override fun onStop() {
        super.onStop()
        saveData()
    }
    fun saveData() {
        try {
            val output : FileOutputStream? = activity?.openFileOutput( name: "data", AppCompatActivity.MODE_PRIVATE)
            val objectOutputStream = ObjectOutputStream(output)
            objectOutputStream.writeObject(game)
            objectOutputStream.close()
            output?.close()
        }catch (e: Exception) {
            e.printStackTrace()
        }
    }
    fun loadData(): CardMatchingGame? {
        try {
            val input : FileInputStream? = activity?.openFileInput( name: "data")
            val objectInputStream =  ObjectInputStream(input)
            val game : CardMatchingGame = objectInputStream.readObject() as CardMatchingGame
            objectInputStream.close()
            input?.close()
            return game
        } catch (e: Exception) {
            e.printStackTrace()
            return null
        }
    }
    companion object {
        lateinit var game: CardMatchingGame
        @JvmStatic
        fun newInstance() : GameFragment = GameFragment()
    }
}
```

4.Card.kt

```
import java.io.Serializable

class Card(private var suit: String, private var rank: String, var isChosen:Boolean = false,
    var isMatched:Boolean = false): Serializable  {
    companion object {
```

5.CardMatchingGame.kt

```
import java.io.Serializable

class CardMatchingGame(val count: Int):Serializable {
    var score = 0
```